INSIGHT.AI

Local Document Intelligence

Document	System Architecture Report
Version	1.0
Audience	Client - AKA Studio
Project	Project INSIGHT.AI

1. Document Purpose

This document describes the technical architecture of the project INSIGHT.AI prototype developed for AKA Studio. It is intended as a reference for developers and technical stakeholders involved in the current implementation or future development of the system. The document covers the system's core components, pipeline design, data flow, security considerations and known limitations, providing a comprehensive technical overview of the prototype as delivered at the end of the project.

2. System Overview

INSIGHT.AI is a locally hosted AI-powered document assistant built with Retrieval-Augmented Generation (RAG) architecture. The system allows the user to point at a folder of PDFs, Word, or Excel files, it indexes the contents and can query them using natural language questions in a chat interface, receiving accurate responses generated by a locally hosted large language model. All processing runs completely locally on the user's computer with no need of specific characteristics. At no stage does any document content, query, or response ever get sent out to the cloud.

3. Architecture Diagram

The following diagram shows the indexing pipeline, query pipeline and index management layer.

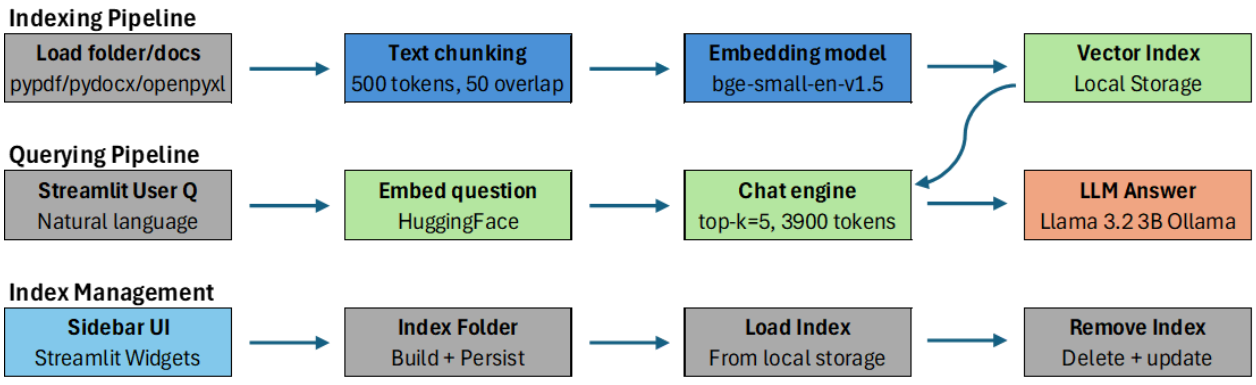

Image 1. Architecture diagram

4. Component Description

The system is built on a Retrieval-Augmented Generation (RAG) architecture, which combines a document retrieval pipeline with a locally hosted LLM to generate accurate responses grounded strictly in the provided documents. The following describes each core component of the system.

Document Ingestion and Processing

The document ingestion process reads and extracts text content from supported file formats, including PDF, Word and Excel. PDF text extraction is handled by the pypdf library, Word files are processed using python-docx, and Excel files are read using openpyxl with the data_only flag to ensure bringing the results from formulas rather than raw expressions. To improve model identification of document sources, the filename is attached to the extracted text of each document before processing. Scanned PDFs and xls Excel 97-2003 files are not currently supported as a known limitation in the prototype.

Text Chunking

Once extracted, document text is split into smaller segments using the LlamaIndex SentenceSplitter function, configured with a chunk size of 500 tokens and an overlap of 50 tokens. This overlap ensures that contextual continuity is maintained across chunk boundaries, reducing the risk of losing relevant information.

Embedding Generation

Each text chunk is converted into a high-dimensional vector representation using the BAAI/bge-small-en-v1.5 embedding model, accessed locally through the HuggingFace library. These vector embeddings capture semantic meaning of each chunk, enabling similarity-based retrieval that goes beyond simple keyword matching. This embedding model runs locally, ensuring that no document content is transmitted externally during the process.

Vector Index and Storage

The generated embeddings are locally stored in a LlamaIndex VectorStoreIndex. By having these vectors stored locally on the user's computer, it means that documents do not need to be reprocessed on subsequent sessions, the index is loaded directly from disk. For persistence, a JSON-based registry file is maintained alongside the indexes to track which folders have been indexed and which documents they contain.

Query Engine and Language Model

User queries are processed through LlamaIndex chat engine, configured with a similarity_top_k value of 5, meaning the five most semantically relevant chunks are retrieved and passed to the language model as context. The language model used is Llama 3.2 3b, accessed locally through the Ollama framework. Ollama manages the local deployment of the model without requiring an internet connection. The engine maintains a conversation memory buffer of 3,900 tokens, allowing users to ask follow-up questions within the same session. A strictly defined system prompt instructs the model to answer only from the provided document context, cite source documents in its responses and explicitly indicate when requested information cannot be found, mitigating the risk of hallucination.

Frontend Interface
The user interface is built using Streamlit and it is structured with a sidebar for index management, including folder indexing, index selection, document visibility, and index deletion, and a main chat area where users can submit natural language queries and receive responses. The dashboard runs locally on and is launched via batch script, requiring no technical knowledge from the end user beyond the initial setup.

5. Pipeline Description

The diagram consists of three components, which are described below.

Indexing Pipeline
This is the process that takes place during a click on "Index Documents" by the user.

- First, the application loads the content of each file in the chosen directory through the appropriate loader:
    - pypdf for PDF files
    - python-docx for .doc files
    - openpyxl for .xls/xlsx files
- Then, the text of the documents loaded is divided into 500 tokens with a 50-token overlap.
- After that, each tokenized text is embedded in the bge-small-en-v1.5 model.
- The created index is locally stored in ~/INSIGHT_AI_storage for future use since the process of indexing is done only once.

Querying Pipeline
This process starts when the user makes a question in natural language within the Streamlit dashboard.

- If a user asks the system any question, the text will be converted into a vector through the same embedding model as was used for indexing.
- Afterward, the system retrieves the top five closest vectors based on similarity from the previously mentioned persisted vector store.
- The system sends both the five vectors and the initial text to LLaMA 3.2.
- The latter reads everything and composes an answer, which appears in the chat window of Streamlit.

Index Management
The index management layer is accessible through the sidebar of the Streamlit dashboard, and includes three main processes providing the user full control over which documents the system can access.

Index Folder: The user provides a local folder path containing the documents to be analysed,which can include PDF, Word or Excel files. Upon clicking ‘Index Documents’, the system executes the complete indexing pipeline. When a folder is reindexed, the existing index directory is deleted and rebuilt from scratch, ensuring that the latest information is stored.

Load Index: Once one or more indexes have been created, the user can select any previously indexed folder from a menu and load it as the active index. The user can also visualize the documents that are included in that specific folder, helping the user to know which documents are going to be used. Upon loading, the system restores the vector index from local storage and initialises the chat engine, making the documents available for querying. The active index is shown in a status indicator at the top of the dashboard at all times.

Remove Index: The user can permanently delete any index that is no longer required, and the registry will be updated, allowing the user to manage storage space and maintain an organised set of indexes. If the removed index was the currently active one, the session is cleared and the user is prompted to load a new index.

6. Security Architecture

The security architecture of INSIGHT.AI is based on a single core principle: all processing occurs entirely on the host machine. No document, user query, embedding or generated response is transmitted to any external service at any point during operation. This design addresses the primary requirements established at the beginning of the project, providing the client with an AI tool that mitigates the risk associated with cloud-based AI platforms.

The system requires an internet connection only during the initial one-time setup to download the multiple technologies used in the project. Once downloaded, all of the components are cached locally and the system operates in a fully offline mode for all subsequent sessions, ensuring that sensitive data remain within the organisation's own infrastructure at all times.

7. Known Limitations

The following limitations have been identified during the development of and testing of the INSIGHT.AI prototype. These represent the boundaries of the current implementation and inform recommendations for future development.

Document Processing

- Scanned PDFs are not supported.
- Protected documents cannot be processed.
- Legacy XLS are not supported.
- PowerPoint files are not currently supported.
- Excel with merged cells, multiple unrelated tables per sheet, or complex formatting may produce inaccurate text extraction.

Retrieval and Query

- When querying complete annual datasets from Excel files, the model may truncate output before completing all entries due to the output token limit of Llama 3.2 3B, users are advised to query by quarter or half-year for complete results.

- Document version comparison is limited, when two versions of a document are indexed together, the retrieval engine may not consistently cover chunks from both versions simultaneously.

Language Model

- Llama 3.2 3B has limited mathematical reasoning capability, complex calculations should be independently verified.
- Response times range from 20 seconds for simple queries to over 90 seconds for complex queries on larger document sets.
- Conversation memory is limited to 3900 tokens, very long sessions may lose earlier context.

Infrastructure

- The system requires a 16GB RAM, performance degrades on lower-specification machines.
- The system is designed for single-user local deployment and does not support multi-user or networked environments.

8. Future Improvements

The following improvements are recommended for future development phases, prioritised based on their potential impact on system performance and user experience.

Short-term improvements - High priority

Improvement	Description
Upgrade to a larger language model (7B-8B)	Replacing Llama 3.2 3b with a model such as Llama 3.1 8b or Mistral 7b would improve numerical reasoning and produce more accurate responses for complex queries
Automatic change detection	Rather than reindexing all documents on every update, the system could detect which files have changed and only reprocess those, improving efficiency for large document sets

Medium-term improvements

Improvement	Description
FastAPI and React frontend	Replacing Streamlit with a FastAPI backend and React frontend would enable multi-user support and a more polished user interface
Advanced vector store	Replacing the built-in LlamaIndex vector store

	with a dedicated vector database would improve scalability to thousands of documents and support concurrent access
PowerPoint Support	Enabling the ingestion of PPTX files using python-pptx would expand the range of supported formats
Multi-index querying with conversation memory	Enabling simultaneous querying across multiple indexes while preserving conversation history
Improved document parsing	Using a more sophisticated PDF parser would preserve table structure and formatting, improving extraction quality for complex documents

These improvements build directly on the modular RAG architecture of the current prototype. The system was designed so that individual components, including the language model, embedding model, vector store and front end can be upgraded independently without requiring a complete system design.